

Building the Ibex chip on IHP SG13G2 with Synopsys

The whole flow from the repository, step by step

This document walks the whole flow that builds the chip, from a fresh clone of the repository to the package IHP takes for its shuttle. Each step gives the command, the check that must pass and a screenshot. Every screenshot comes from a fresh build of the repository in a run named test1. The tutorial, Ibex tutorial: RTL to GDSII with Synopsys, explains the ideas behind each step.

Item	Value
Document	Building the Ibex chip on IHP SG13G2 with Synopsys
Version	2.0, 2026-10-05
Author	Cesar Valentin
Repository	https://github.com/XxMasterepicxX/ibex-sg13g2-chip
Chip	flash_chip: Ibex, two 4 KiB memories, a serial loader, a scan chain and 25 pads
Kit	IHP SG13G2, 130 nm
Tools	Fusion Compiler, StarRC, PrimeTime SI, Formality, PrimePower, TestMAX, VCS, SpyGlass, RedHawk-SC, and KLayout with IHP's rules

Contents

Before you start.....	3
Get the repository and set it up.....	3
What is in the repository.....	4
Stage 1: Make a run.....	4
Stage 2: The whole build in one command.....	5
Stage 3: Synthesis and scan.....	5
Stage 4: Place and route.....	6
4.1 Floorplan and power grid.....	6
4.2 Placement and clock tree.....	7
4.3 Routing.....	9
4.4 Final checks and files.....	11
Stage 5: Finish the chip.....	11
Stage 6: Signoff.....	13
6.1 Wire delays, StarRC.....	14
6.2 Timing, PrimeTime.....	14
6.3 Logic, power, DRC and LVS.....	15
6.4 The checker.....	15
Stage 7: Margin rounds.....	16
Stage 8: Proving it works.....	17
8.1 Programs on the gates.....	17
8.2 Scan test.....	18
8.3 Clock crossings.....	19
8.4 Voltage drop and electromigration.....	19
Stage 9: Review records and final signoff.....	20
Stage 10: The release package.....	22
Results of the fresh build.....	23
Command summary.....	23

Before you start

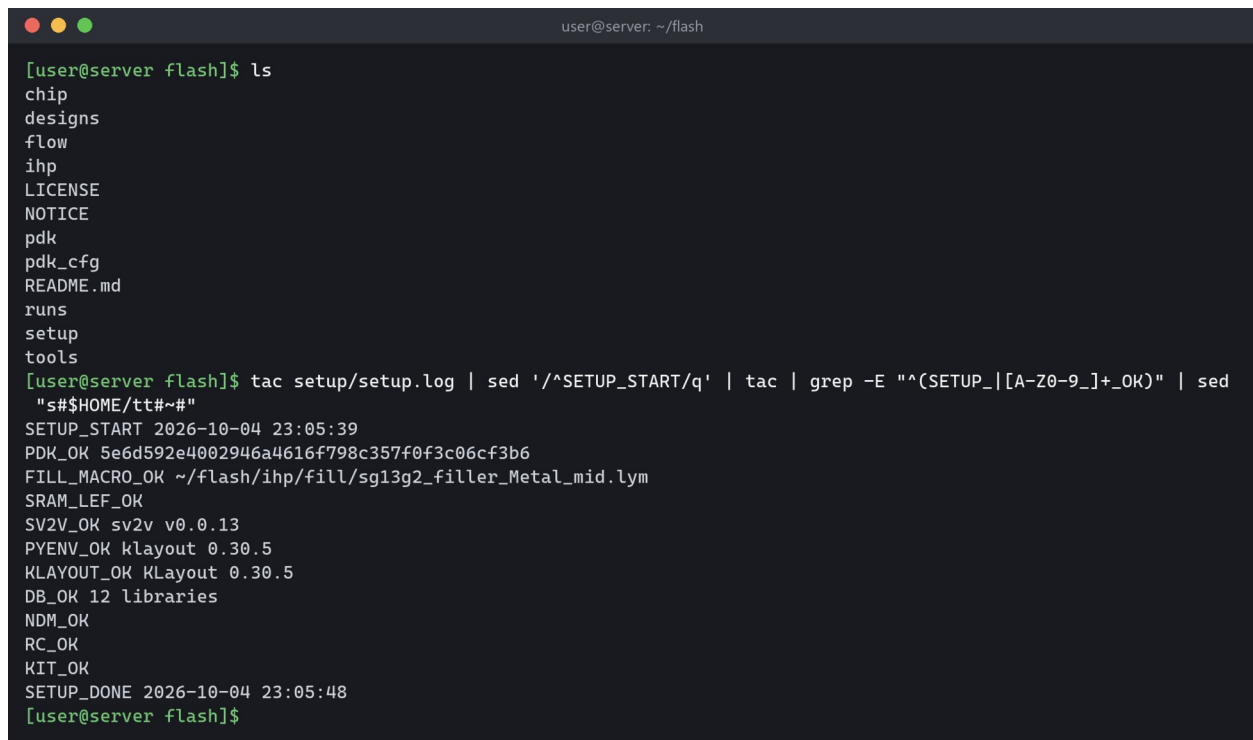
- Log in to a Linux server where the Synopsys and Ansys tools live under /apps. Off campus, connect to the VPN first.
- The scripts load the tools for you. If you start a tool by hand, run `source /apps/settings` first.
- Fusion Compiler licenses are shared with other users. When none is free, the build waits and tries again.
- Long steps run for hours. Start them under `nohup` so they keep going after you log out.

Get the repository and set it up

Clone the repository into `~/flash` and run its setup once. Every script finds its files in `~/flash`, so do not clone it anywhere else.

```
git clone https://github.com/XxMasterepicxX/ibex-sg13g2-chip.git ~/flash
bash ~/flash/setup/setup.sh
```

Setup takes about two hours. It downloads IHP's kit, builds KLayout, and installs `sv2v` and the Python packages. Then it builds the libraries and wire models the Synopsys tools read, from IHP's kit and with your own licenses. Those files are not in the repository, because they are in Synopsys formats. Each step prints a line ending in `_OK`. If setup stops, the last line names the step, and that step's log says why. Run `setup.sh` again after fixing it. Every step that already passed skips itself.

A terminal window titled 'user@server: ~/flash' showing the output of a setup script. The first command is 'ls', listing files like 'chip', 'designs', 'flow', 'ihp', 'LICENSE', 'NOTICE', 'pdk', 'pdk_cfg', 'README.md', 'runs', 'setup', and 'tools'. The second command is a complex pipeline to parse the 'setup.log' file, showing a list of steps and their completion status, all ending in '_OK'. The final line shows 'SETUP_DONE' at 2026-10-04 23:05:48.

```
user@server: ~/flash
[user@server flash]$ ls
chip
designs
flow
ihp
LICENSE
NOTICE
pdk
pdk_cfg
README.md
runs
setup
tools
[user@server flash]$ tac setup/setup.log | sed '/^SETUP_START/q' | tac | grep -E "^(SETUP_|[A-Z0-9_]+_OK)" | sed
"s#$$HOME/##"
SETUP_START 2026-10-04 23:05:39
PDK_OK 5e6d592e4002946a4616f798c357f0f3c06cf3b6
FILL_MACRO_OK ~/flash/ihp/fill/sg13g2_filler_Metal_mid.lym
SRAM_LEF_OK
SV2V_OK sv2v v0.0.13
PYENV_OK klayout 0.30.5
KLAYOUT_OK KLayout 0.30.5
DB_OK 12 libraries
NDM_OK
RC_OK
KIT_OK
SETUP_DONE 2026-10-04 23:05:48
[user@server flash]$
```

Figure 1. The repository after setup, and the end of `setup.log`. This was a second run of `setup.sh`, so each step only checked its result.

Setup is done when `~/flash/setup/setup.log` ends with `SETUP_DONE`.

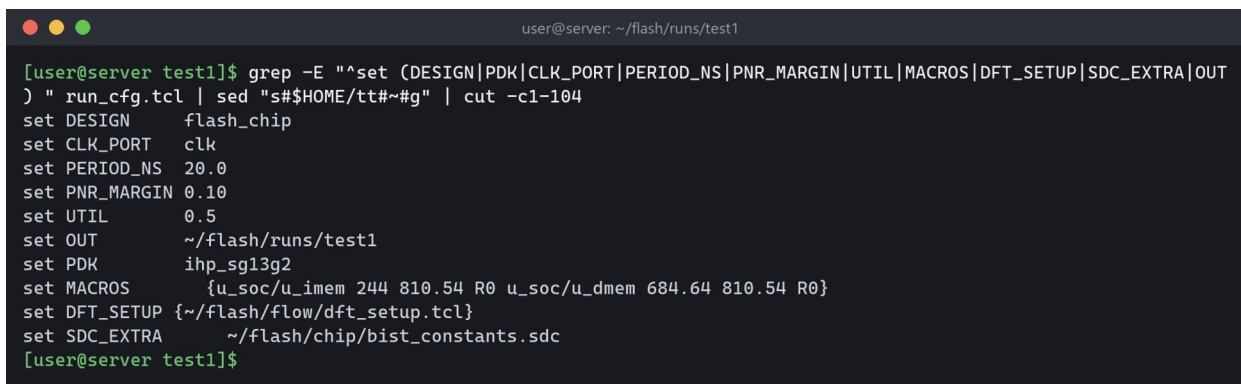
What is in the repository

Folder	What it holds
setup/	the one-time setup
flow/	the Fusion Compiler flow <code>fc_flow.tcl</code> , <code>run_fc.sh</code> , <code>signoff.sh</code> , the checker <code>check.py</code> , chip finishing <code>finish.sh</code> and the fix loop <code>eco_fixed.sh</code>
pdk_cfg/ihp_sg13g2/	every fact about the IHP kit: layers, cells, corners, pads, bond pads and fill
ihp/	the scripts that build the Synopsys libraries, and the wire models
designs/flash_soc/	the RTL, the testbench and the test programs
chip/	the one-command build, margin rounds, proofs, review records and the release package
runs/<name>/	one folder per build, made by the scripts

Stage 1: Make a run

```
bash ~/flash/chip/new_run.sh my_chip
```

This makes the folder `~/flash/runs/my_chip` with one settings file, `run_cfg.tcl`, copied from `chip/run_cfg.tcl` with the paths set to your home folder. Every later step reads it.



```
user@server: ~/flash/runs/test1

[user@server test1]$ grep -E "^set (DESIGN|PDK|CLK_PORT|PERIOD_NS|PNR_MARGIN|UTIL|MACROS|DFT_SETUP|SDC_EXTRA|OUT)" run_cfg.tcl | sed "s#\$HOME/tt#~#g" | cut -c1-104
set DESIGN      flash_chip
set CLK_PORT     clk
set PERIOD_NS   20.0
set PNR_MARGIN  0.10
set UTIL        0.5
set OUT         ~/flash/runs/test1
set PDK         ihp_sg13g2
set MACROS       {u_soc/u_imem 244 810.54 R0 u_soc/u_dmem 684.64 810.54 R0}
set DFT_SETUP    {~/flash/flow/dft_setup.tcl}
set SDC_EXTRA    ~/flash/chip/bist_constants.sdc
[user@server test1]$
```

Figure 2. The main settings of the run. *OUT* must be the run folder itself, and *run_fc.sh* refuses to start if it is not.

Setting	What it does
DESIGN	the top module, <code>flash_chip</code> : the core inside its pad ring
PERIOD_NS	the clock period signed off, 20 ns or 50 MHz
PNR_MARGIN	0.10 makes Fusion Compiler aim for a period 10% shorter, 18 ns, to leave room
UTIL	cells fill half of the core area at the start. The rest is room for wires
MACROS	where the two memories go, in μm from the lower left corner
DFT_SETUP	how to build the scan chain
SDC_EXTRA	holds the memories' unused self-test clocks at 0 for timing

The chip must work at three corners. Each corner pairs a voltage and temperature with the matching library and wire model.

Corner	Voltage	Temperature	Checked mainly for	Wire model
slow	1.08 V	125 C	setup	worst
typ	1.2 V	25 C	power	typical
fast	1.32 V	-40 C	hold	best

Stage 2: The whole build in one command

```
nohup bash ~/flash/chip/make_chip.sh my_chip > /dev/null 2>&1 &  
tail -f ~/flash/runs/my_chip/make_chip.log
```

make_chip.sh makes the run if it does not exist and runs stages 3 to 8 in order. Each step must pass before the next starts. If one fails, the log names it and says where to look. After a fix, run the same command again. Steps that already passed are skipped. It stops when the review records are ready for a person to sign, in stage 9.

The stages below show each step it runs, with the command to run that step by hand and the check it must pass. In every command, RUN is the run folder:

```
RUN=~/flash/runs/my_chip  
export RUN_CFG=$RUN/run_cfg.tcl FLASH_T00LS=2026
```

Do not trust FC_EXIT=0 in a Fusion Compiler log. Fusion Compiler prints it even after an error. A session worked only if its log has FLASH_FLOW_DONE, or FLASH_STOPPED_AFTER when it was asked to stop.

Stage 3: Synthesis and scan

```
STOP_AFTER=synth ~/flash/flow/run_fc.sh $RUN
```

Fusion Compiler reads the RTL and builds it from IHP's standard cells. Then it inserts the scan chain, which lets a tester load and read every flip-flop. The session stops after synthesis so you can check the result before placing anything.

```
user@server: ~/flash/runs/test1

[user@server test1]$ grep -h "^FLASH_IHP_PADS\\|^FLASH_STAGE_DONE\\|^FLASH_STOPPED" fc_setup.log
FLASH_STAGE_DONE setup
FLASH_IHP_PADS_PRE_SYNTH 25
FLASH_IHP_PADS_POST_DFT 25
FLASH_STAGE_DONE synth
FLASH_STOPPED_AFTER synth
[user@server test1]$ grep -E "Number of (cells|sequential cells|macros)|Total cell area" rpt/area_synth.rpt
Number of cells: 12222
Number of sequential cells: 1866
Number of macros/black boxes: 2
Total cell area: 746487.715
[user@server test1]$
```

Figure 3. The synthesis log and the area report. All 25 pads are still there after scan insertion.

Synthesis is done when the log shows FLASH_IHP_PADS_POST_DFT 25. The netlist has 12,222 cells, 1,866 of them flip-flops, and the two memories.

Stage 4: Place and route

```
START_AT=floorplan ~/flash/flow/run_fc.sh $RUN
```

The second Fusion Compiler session runs the rest of the build: floorplan, power grid, placement, clock tree, routing and the final checks. It saves the design after each step. START_AT restarts from a saved step. The steps are floorplan, pg, place, cts, route and final.

4.1 Floorplan and power grid

The floorplan sets the die size, places the 25 pads in their ring and puts the two memories at the top of the core. The power grid then brings VDD and VSS from the pad ring to every row of cells.

```
user@server: ~/flash/runs/test1

[user@server test1]$ grep -h "^FLASH_MACROS\\|^FLASH_UNPLACED_PORTS\\|^FLASH_IO_CORE_STRAPS\\|^FLASH_PG_FLOATING" fc_floorplan.log
FLASH_MACROS u_soc/u_imem u_soc/u_dmem
FLASH_UNPLACED_PORTS 0
FLASH_IO_CORE_STRAPS VDD 36
FLASH_IO_CORE_STRAPS VSS 32
FLASH_PG_FLOATING_WIRES 0 0
[user@server test1]$ grep -E "floating (wires|vias|hard macros|I/O pads)" rpt/pg_conn_pg.rpt | sort | uniq -c
  2 Number of floating hard macros: 0
  2 Number of floating I/O pads: 0
  2 Number of floating vias: 0
  2 Number of floating wires: 0
[user@server test1]$
```

Figure 4. The floorplan and power grid reports. Both memories are placed and no input or output is left unplaced. Nothing in the power grid floats.

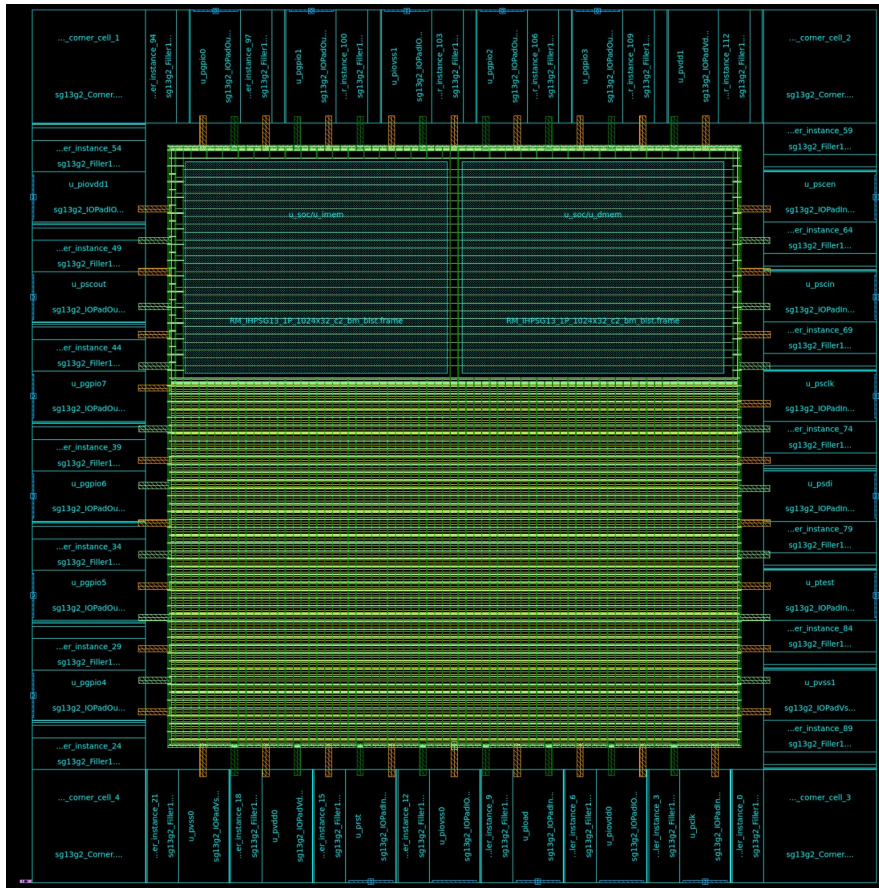


Figure 5. The chip after the power grid. The 25 pads and their fillers form the ring, the two memories sit at the top of the core, and the grid's stripes cover the rows below them.

The floorplan and power grid are done when FLASH_UNPLACED_PORTS is 0 and the power grid report shows no floating wires, vias, memories or pads.

4.2 Placement and clock tree

Placement puts every cell on a row and improves timing as it goes. The clock tree then carries the clock to every flip-flop through buffers, so they all see it at nearly the same moment.

```

user@server: ~/flash/runs/test1

[user@server test1]$ grep "succeeded\\|failed" rpt/legality_place.rpt | tail -1
check_legality succeeded.
[user@server test1]$ grep -E "(Clock /|Skew Group|clk |All Clocks)" rpt/clock_qor.rpt | head -4 | cut -c1-120
Clock /                               Attrs      Sinks Levels   Clock      Clock      Clock      Max      Global  T
rans DRC
Skew Group
Count
clk                                     M,D        1865    10      461  3449.174  17880.020   1.091   0.531
clk                                     M,D        1865    10      461  3449.174  17880.020   1.091   0.531
0
[user@server test1]$ grep -h "FLASH_CTS_ROUTE_DRC" fc_floorplan.log
FLASH_CTS_ROUTE_DRC 189
[user@server test1]$

```

Figure 6. A legal placement, and the clock tree: 1,865 flip-flops reached through 10 levels of buffers. The two clk lines are the normal and scan modes.

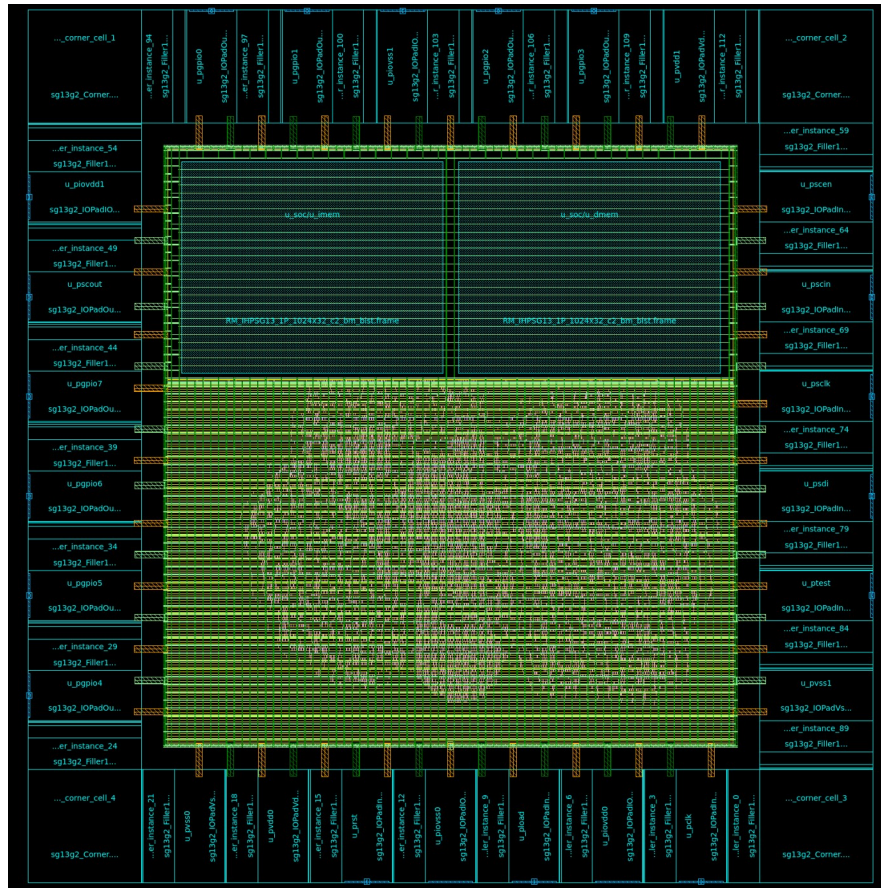


Figure 7. After placement. Every standard cell sits on a row in the core below the memories.

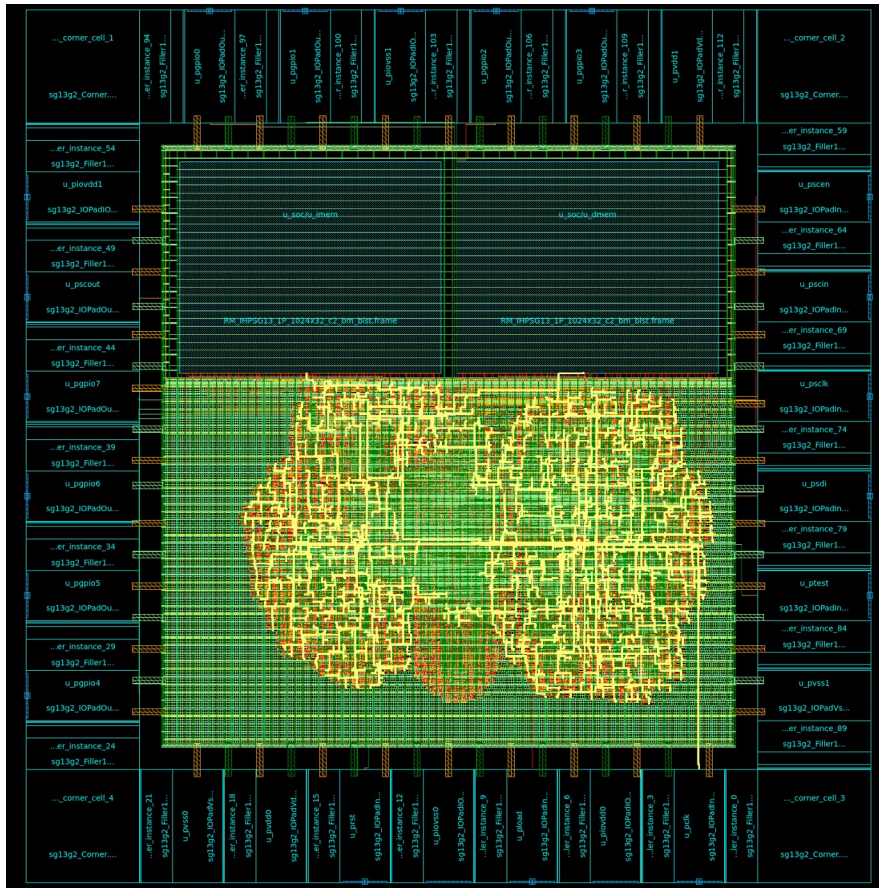


Figure 8. After the clock tree, with the clock nets in yellow. They spread from the clock pad to every flip-flop.

The clock tree's early wiring check counts some errors, 189 here. The log prints the count as FLASH_CTS_ROUTE_DRC and the flow goes on, because routing redraws those wires. The check that counts is the one after routing.

4.3 Routing

Routing turns every connection into metal wires and vias on Metal2 to Metal5, then repairs any rule errors it made.

```

user@server: ~/flash/runs/test1

[user@server test1]$ grep -E "^Total number of (nets|open nets|DRCs|antenna)" rpt/check_routes.rpt | awk '!s[$0]++'
Total number of nets = 15673, of which 0 are not extracted
Total number of open nets = 0, of which 0 are frozen
Total number of DRCs = 0
Total number of antenna violations = 0
Total number of DRCs on Metal3 and below = 0
[user@server test1]$ grep -E "^Total number of (short|open|floating)" rpt/check_lvs.rpt
Total number of short violations is 0.
Total number of open nets is 0.
Total number of floating route violations is 0.
[user@server test1]$ grep -h "^FLASH_ROUTE_CLEANUP|^FLASH_FLOW_DONE" fc_floorplan.log
FLASH_ROUTE_CLEANUP pass=1 drcs=0 signal_nets=0
FLASH_FLOW_DONE
[user@server test1]$

```

Figure 9. The routing checks. There are no open nets, routing errors, antenna errors or shorts.

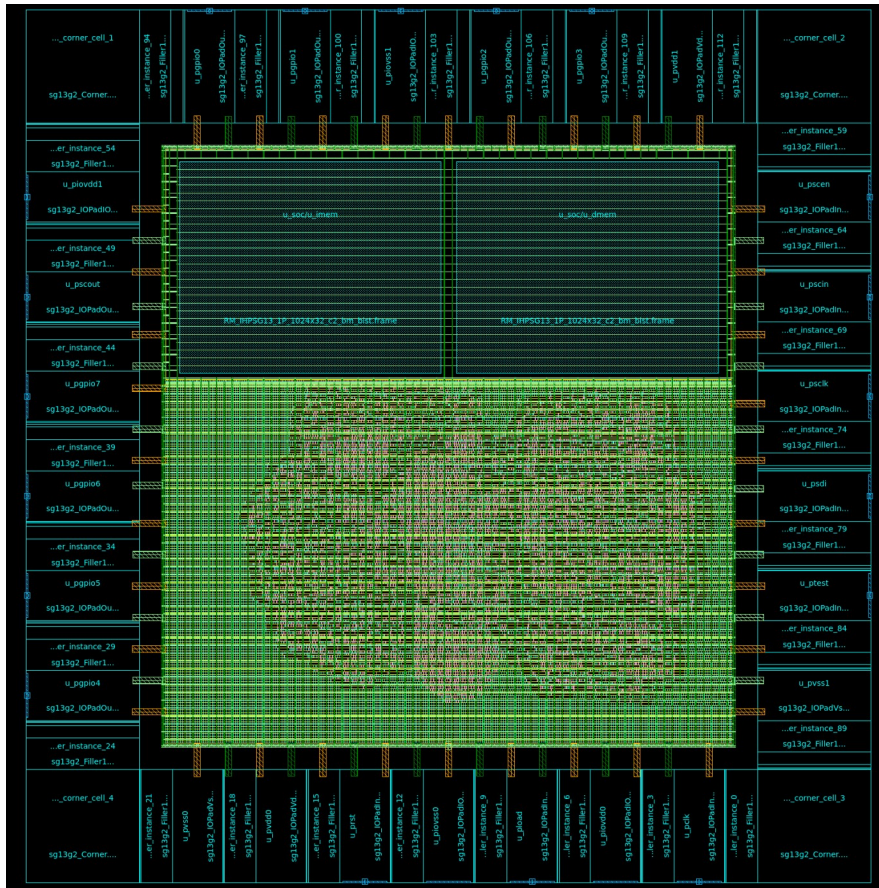


Figure 10. After routing. Signal wires on Metal2 to Metal5 cover the core.

Routing is done when check_routes reports 0 DRCs and 0 open nets, and the short check reports 0 shorts.

4.4 Final checks and files

```
user@server: ~/flash/runs/test1

[user@server test1]$ sed -n '/^Context/,/^Miscellaneous/p' rpt/final_qor.rpt | grep -v "^---\|^Misc"
Context                                WNS          TNS          NVE
func_slow      (Setup)        -0.059      -0.288          9
func_typ       (Setup)         6.116         0.000          0
Design        (Setup)        -0.059      -0.288          9

func_fast      (Hold)         0.001         0.000          0
func_typ       (Hold)         0.043         0.000          0
shift_fast     (Hold)         0.001         0.000          0
shift_slow     (Hold)         0.098         0.000          0
shift_typ      (Hold)         0.043         0.000          0
Design        (Hold)         0.001         0.000          0
[user@server test1]$ ls out
flash_chip.def
flash_chip.floorplan.svf
flash_chip.gds
flash_chip.Internal_scan.spf
flash_chip.Internal_scan.synth.spf
flash_chip.pg.v
flash_chip.setup.svf
flash_chip.synth.v
flash_chip.v
[user@server test1]$
```

Figure 11. Fusion Compiler's own timing at the end, and the files it writes to out/.

Fusion Compiler judges timing against its 18 ns target, so a small negative slow setup slack here is normal. Signoff in stage 6 judges the real 20 ns period with separate tools. Hold must already be positive everywhere. Stage 4 is done when the log ends with FLASH_FLOW_DONE.

Stage 5: Finish the chip

```
METAL_FILL=mid bash ~/flash/flow/finish.sh $RUN 75 bondpads
```

finish.sh adds what the factory needs around the routed design. Bond pads go on every pad. A seal ring goes around the edge, 75 μm out. Metal fill goes into every empty area, because the factory polishes each layer flat and needs even metal density. Then it runs whole-chip DRC and IHP's pre-check, the rule set IHP runs before it accepts a design.

make_chip.sh first runs a check-only signoff, signoff.sh \$RUN check. That records the signoff inputs, which the chip DRC result is tied to. Run it too if you run finish.sh by hand.

```
user@server: ~/flash/runs/test1

[user@server test1]$ grep -h "^FLASH_IHP_PADS_POST_DFT\|^FLASH_FLOW_DONE" fc_setup.log fc_floorplan.log
FLASH_IHP_PADS_POST_DFT 25
FLASH_FLOW_DONE
[user@server test1]$ cat chip/density.txt
bbox (0,0;1708,1753) area 2994124 um2
Activ      47.2%   0:29.6% 22:17.6%
GatPoly    17.7%   0:7.0% 22:10.7%
Metal1     49.7%   0:29.8% 22:19.9%
Metal2     44.6%   0:8.3% 22:36.2%
Metal3     53.5%   0:31.3% 22:22.2%
Metal4     54.6%   0:32.4% 22:22.2%
Metal5     55.0%   0:28.1% 22:26.9%
TopMetal1  51.5%   0:24.8% 22:26.7%
TopMetal2  50.5%   0:21.5% 22:29.1%
[user@server test1]$ grep -h "Number of DRC errors" chip/drc.log chip/precheck.log
Number of DRC errors for maximum rule set: 0
Number of DRC errors for maximum rule set: 0
[user@server test1]$
```

Figure 12. The pad count, the end of the flow, the metal density and both DRC results. Every metal is inside IHP's limits and both DRC runs found 0 errors.

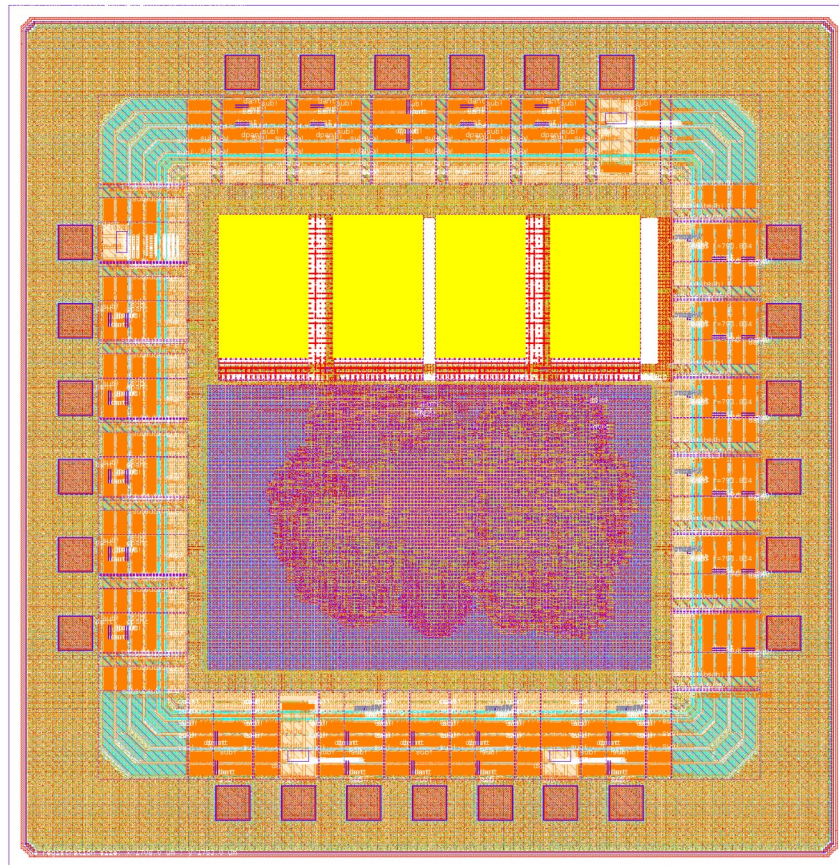


Figure 13. The finished chip in KLayout, 1708 by 1753 μm . The seal ring runs around the edge, the 25 square bond pads sit on the pads, the two memories are yellow, and fill covers every empty area.

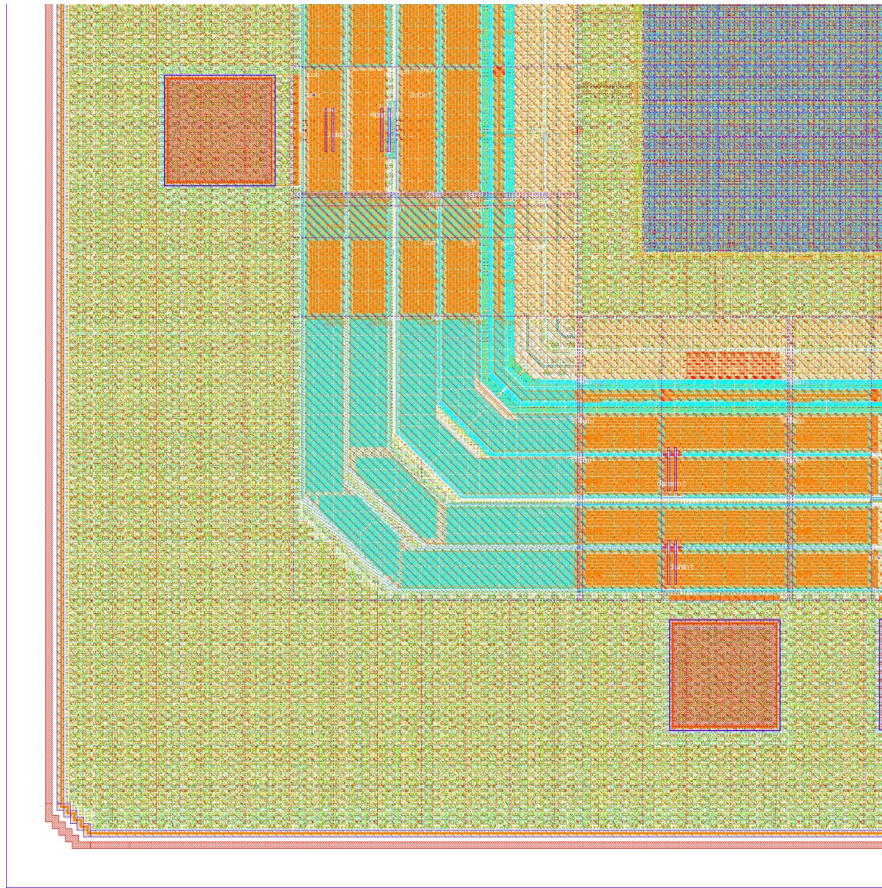


Figure 14. The lower left corner, 560 μm across. From the outside in: the seal ring, two bond pads, the pad ring's corner cell with its supply rails, and the edge of the core. The fine pattern everywhere else is metal fill.

IHP requires 35% to 60% metal on Metal1 to Metal5 and 25% to 70% on TopMetal1 and TopMetal2. Stage 5 is done when every layer in chip/density.txt is inside those limits and both DRC runs report 0 errors.

Stage 6: Signoff

```
export RC_CMAX_GRID=~/.flash/ihp/rc/w6_spec_sensitivity/rcmax/sg13g2_spec_rcmax.nxtgrd
export RC_CMIN_GRID=~/.flash/ihp/rc/w6_spec_sensitivity/rcmin/sg13g2_spec_rcmin.nxtgrd
~/.flash/flow/signoff.sh $RUN
```

Signoff checks the design with tools that did not build it. StarRC extracts every wire at the typical, worst and best wire models. PrimeTime checks timing, crosstalk and electrical limits at three corners in normal and scan mode, six runs in all. Formality proves the netlist does the same logic as the RTL. PrimePower adds up the power. KLayout runs IHP's DRC and LVS on the block and LVS on the whole chip. Last, check.py reads every report and writes one line per check to signoff/CHECK.txt.

The two exports point at the worst and best wire models setup built. IHP publishes only a typical model, so these come from the ranges in IHP's process specification, as ihp/rc/w6_spec_sensitivity/DERIVATION.md describes. Signoff took about two and a half hours on the fresh build.

6.1 Wire delays, StarRC

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ tail -qn1 starrc/starrc.log starrc_cmax/starrc.log starrc_cmin/starrc.log
STARRC_EXIT=0
STARRC_EXIT=0
STARRC_EXIT=0
[user@server signoff]$ stat -c '%s bytes  %n' starrc*/flash_chip.spef
14056240 bytes  starrc_cmax/flash_chip.spef
14088884 bytes  starrc_cmin/flash_chip.spef
14054721 bytes  starrc/flash_chip.spef
[user@server signoff]$
```

Figure 15. StarRC finished at the typical, worst and best wire models. Each run writes one SPEF file with the resistance and capacitance of every net.

6.2 Timing, PrimeTime

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ for c in slow typ fast slow_shift typ_shift fast_shift; do printf "%-11s" $c; grep -h "^FLASH_PT corner" pt_$c/pt.log | head -1 | grep -o "setup_wns=[-0-9.]* hold_wns=[-0-9.]*"; done
slow      setup_wns=1.142242 hold_wns=0.138854
typ       setup_wns=7.836167 hold_wns=0.084181
fast      setup_wns=11.673019 hold_wns=0.036749
slow_shift setup_wns=1.324516 hold_wns=0.138895
typ_shift  setup_wns=7.864738 hold_wns=0.081989
fast_shift setup_wns=11.686943 hold_wns=0.036704
[user@server signoff]$ for c in slow typ fast; do grep -h "^FLASH_PT_NOISE" pt_$c/pt.log; done
FLASH_PT_NOISE corner=slow worst_slack=0.03 violators=0
FLASH_PT_NOISE corner=typ worst_slack=0.04 violators=0
FLASH_PT_NOISE corner=fast worst_slack=0.03 violators=0
[user@server signoff]$ grep -c VIOLATED pt_slow/violators.rpt pt_typ/violators.rpt pt_fast/violators.rpt
pt_slow/violators.rpt:1
pt_typ/violators.rpt:1
pt_fast/violators.rpt:1
[user@server signoff]$
```

Figure 16. The six PrimeTime runs. Setup and hold pass in every corner and mode, and crosstalk noise has no violators. Each corner lists one electrical violation.

Slow setup has +1.14 ns to spare. Fast hold has only +0.037 ns, which passes but leaves little room for the small differences between real chips. The one electrical violation at every corner is the same pin. The flip-flop `u_soc/sc1k_s_reg[2]` uses only its inverted output, so its other output drives nothing and is under its smallest allowed load. Stage 7 fixes both.

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ awk '/Startpoint/{n++; n==1' pt_slow/setup.rpt | grep -E "Startpoint|Endpoint|Derating|data arrival time|data required time|slack \\" | sed 's/ */ /g' | awk '!s[$0]++' | cut -c1-104
Startpoint: u_soc/u_core/u_ibex_core/if_stage_i/instr_rdata_id_o_reg[18] (rising edge-triggered flip-fl
Endpoint: u_soc/u_core/gen_regfile_ff.register_file_i/g_plain_rf.g_rf_flops[4].rf_reg_q_reg[27] (rising
Max Data Paths Derating Factor : 1.05
Min Clock Paths Derating Factor : 0.95
Max Clock Paths Derating Factor : 1.05
data arrival time 19.20
data required time 20.34
data arrival time -19.20
slack (MET) 1.14
[user@server signoff]$
```

Figure 17. The worst setup path at the slow corner. The data arrives at 19.20 ns and is needed by 20.34 ns. The derating factors are the 5% penalty PrimeTime adds on both sides.

6.3 Logic, power, DRC and LVS

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ grep -E "Verification (SUCCEEDED|FAILED)|Passing compare points|Failing compare points" formality/fm.log
Verification SUCCEEDED
2188 Passing compare points
[user@server signoff]$ grep -E "Switching Power|Internal Power|Leakage Power|^Total Power" primepower/power.rpt
Net Switching Power = 1.063e-03 (30.82%)
Cell Internal Power = 2.382e-03 (69.07%)
Cell Leakage Power = 3.743e-06 ( 0.11%)
Total Power = 3.449e-03 (100.00%)
[user@server signoff]$
```

Figure 18. Formality proves all 2,188 compare points of the netlist match the RTL. PrimePower puts the chip at 3.45 mW.

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ grep -h "Number of DRC errors" drc.log
Number of DRC errors for maximum rule set: 0
[user@server signoff]$ grep -h "Congratulations" lvs.log lvs_chip/lvs.log | sed 's/^.*INFO : //'
Congratulations! Netlists match.
Congratulations! Netlists match.
[user@server signoff]$ grep -E "^(drc|lvs|lvs_completion|chip_lvs_completion) " CHECK.txt | cut -c1-104
drc PASS klayout violations=0 recommended_only=0 chip_level=GFi
lvs PASS netlists match
lvs_completion PASS latest match verdict and successful exit required
chip_lvs_completion PASS latest match verdict and successful exit required
[user@server signoff]$
```

Figure 19. KLayout with IHP's rules. Block DRC finds 0 errors. LVS matches for the block and for the whole chip with its pad ring.

6.4 The checker

```
user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ grep -v " PASS " CHECK.txt | cut -c1-104
reports_atpg FAIL missing=signoff/atpg/evidence.json,signoff/atpg/summary
provenance_atpg MISSING evidence missing; no runtime fingerprint
timing_constraints_slow FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_slow FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_slow FAIL missing reviewed timing coverage contract
timing_constraints_slow_shift FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_slow_shift FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_slow_shift FAIL missing reviewed timing coverage contract
timing_constraints_typ FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_typ FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_typ FAIL missing reviewed timing coverage contract
timing_constraints_typ_shift FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_typ_shift FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_typ_shift FAIL missing reviewed timing coverage contract
timing_constraints_fast FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_fast FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_fast FAIL missing reviewed timing coverage contract
timing_constraints_fast_shift FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_fast_shift FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_fast_shift FAIL missing reviewed timing coverage contract
formality_coverage FAIL passing=2188 excluded=1 clock_gate_latches=0; requires
atpg_coverage FAIL [Errno 2] No such file or directory: '~
SIGNOFF NOT CLEAN: 117 of 139 checks pass
[user@server signoff]$
```

Figure 20. Every check that does not pass yet after the first signoff. 117 of 139 pass.

The failing rows fall into three groups, and none is a design error. The scan test has not run yet, which is stage 8. The review records are not signed yet, which is stage 9. The electrical row at each corner is the one flip-flop output above, which stage 7 fixes. Timing, noise, Formality, DRC and both LVS runs already pass.

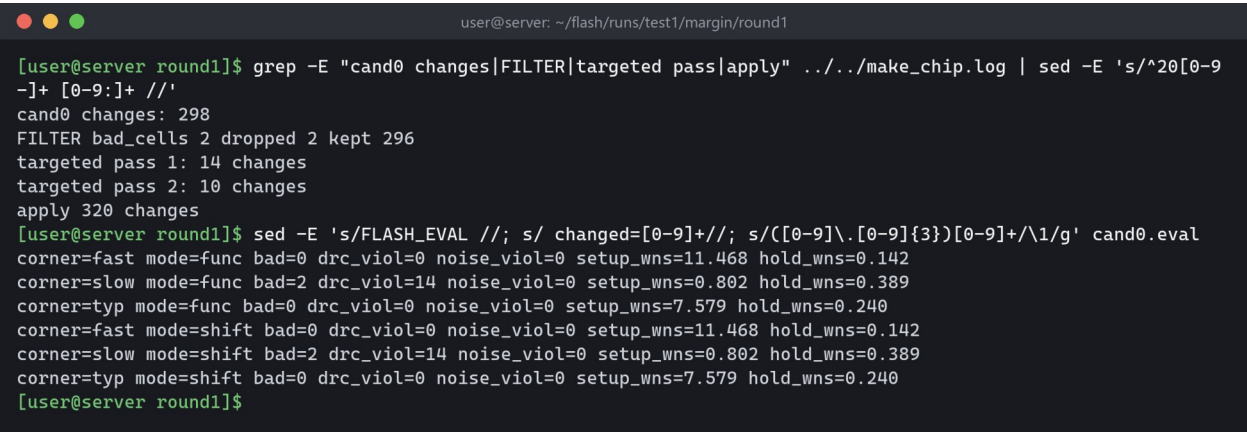
Stage 7: Margin rounds

```
bash ~/flash/chip/margin_fix.sh $RUN
```

The first signoff usually leaves fast hold a little under +0.09 ns, a few noise victims, or a memory output near its load limit. A round fixes them without breaking another corner:

1. PrimeTime proposes noise, load and hold fixes and changes nothing.
2. The list is tested in six PrimeTime runs, three corners in two modes. A changed cell that leaves a path under +0.6 ns at slow or +0.2 ns at typ and fast is dropped, and the shorter list is tested again.
3. Targeted fixes cover what is left. A noise victim's driver gets a buffer after it. A driver over its limit gets one size larger. A hold endpoint gets a delay cell in front of it.
4. Fusion Compiler applies the list only if slow setup keeps +0.6 ns and fast hold reaches +0.09 ns. The chip is finished and signed off again, and the round is undone if any check that passed before fails.

make_chip.sh runs up to three rounds. A kept round prints FLASH_ECO_KEPT. Each round keeps its work in \$RUN/margin/roundN.



```
user@server: ~/flash/runs/test1/margin/round1
[user@server round1]$ grep -E "cand0 changes|FILTER|targeted pass|apply" ../../make_chip.log | sed -E 's/^20[0-9-]+ [0-9:]+ //'
cand0 changes: 298
FILTER bad_cells 2 dropped 2 kept 296
targeted pass 1: 14 changes
targeted pass 2: 10 changes
apply 320 changes
[user@server round1]$ sed -E 's/FLASH_EVAL //; s/ changed=[0-9]+//; s/([0-9]\.[0-9]{3})[0-9]+/\1/g' cand0.eval
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.142
corner=slow mode=func bad=2 drc_viol=14 noise_viol=0 setup_wns=0.802 hold_wns=0.389
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.240
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.142
corner=slow mode=shift bad=2 drc_viol=14 noise_viol=0 setup_wns=0.802 hold_wns=0.389
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.240
[user@server round1]$
```

Figure 21. Round 1 on the fresh build. PrimeTime proposed 298 changes. At the slow corner 2 of them left a path under the floor, and the test dropped both.

With PrimeTime's list alone, fast hold went from +0.037 to +0.142 ns. Dropping the 2 bad cells took it back under the floor, so the targeted fixes ran. The first pass put a delay cell in front of 9 memory inputs and made 5 drivers one size larger. The second pass added a second delay cell on the inputs that were still short, plus one more input.

```

user@server: ~/flash/runs/test1/margin/round1

[user@server round1]$ head -3 targeted1.tcl | cut -c1-80
insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 -new_net_n
insert_buffer [get_pins {u_soc/u_dmem/A_BM[8]}] sg13g2_dlygate4sd1_1 -new_net_na
insert_buffer [get_pins {u_soc/u_dmem/A_BM[9]}] sg13g2_dlygate4sd1_1 -new_net_na
[user@server round1]$ grep -h "^size_cell" targeted1.tcl | head -2
size_cell {u_soc/u_core/u_ibex_core/A4312} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4318} {sg13g2_nand2_2}
[user@server round1]$ sed -E 's/FLASH_EVAL //; s/ changed=[0-9]+//; s/([0-9]\.[0-9]{3})[0-9]+/\1/g' cand3.eval
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.113
corner=slow mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=0.802 hold_wns=0.339
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.202
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.113
corner=slow mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=0.802 hold_wns=0.339
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.202
[user@server round1]$ cat gate.txt
MARGIN_GATE PASS
[user@server round1]$

```

Figure 22. The targeted fixes, and the final test of all 320 changes. Fast hold is now +0.113 ns, slow setup keeps +0.802 ns, and no corner has a bad cell, an electrical violation or a noise violation.

Only then does Fusion Compiler apply the list, finish the chip and sign it off again. That takes about as long as stages 5 and 6 together.

```

user@server: ~/flash/runs/test1/margin/round1

[user@server round1]$ grep -E "FLASH_ECO_(APPLIED|LEGALITY|KEPT|REJECTED|FAILED)" apply.log
FLASH_ECO_LEGALITY_PASS
FLASH_ECO_APPLIED 355
FLASH_ECO_KEPT 123 checks pass against 117 before
[user@server round1]$

```

Figure 23. The round's verdict. The placement stayed legal, and 123 checks pass where 117 passed before, so the round is kept.

After round 1, every timing, noise and electrical row passes and fast hold is +0.107 ns. The rows still failing wait for the scan test and the review records. Our test build ran a second, smaller round of 23 changes, because of a bug in how make_chip.sh decided a round was needed. The bug is fixed, and a new build stops after round 1 here.

Stage 8: Proving it works

8.1 Programs on the gates

```
bash ~/flash/chip/gls.sh $RUN
```

VCS runs six test programs on the final netlist with PrimeTime's delays at slow, typ and fast. The program t_multdiv_plant is built to fail. Its failure proves a failure would show. It passed if it prints GLS_PASS.

```

user@server: ~/flash/runs/test1/gls

[user@server gls]$ for c in slow typ fast; do echo "== $c"; cut -c1-96 $c/summary.txt; done
== slow
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
== typ
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
== fast
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
[user@server gls]$

```

Figure 24. Every program at every corner. Each line shows the result the program must give and the result it gave. All five real programs pass with 0x80 on the GPIO pins, and t_muldiv_plant fails as it must, at slow, typ and fast.

```

user@server: ~/flash/runs/test1/gls

[user@server gls]$ grep -E "TB:|TB_RESULT" slow/t_work/sim.log | cut -c1-104
TB: loading 138 words from prog.hex
TB_RESULT PASS gpio=80 time=1966700000
[user@server gls]$ grep -E "TB:|TB_RESULT" slow/t_muldiv_plant/sim.log | cut -c1-104
TB: loading 161 words from prog.hex
TB_RESULT FAIL gpio=a0 time=1266840000
[user@server gls]$ grep -ih "timing violation" fast/*/sim.log | sed -E 's#.*(RM_[^"]+)", ([0-9]+).*\1, line \2
#\' | sort | uniq -c
111846 RM_IHPSG13_1P_1024x32_c2_bm_bist.v, line 164
110135 RM_IHPSG13_1P_1024x32_c2_bm_bist.v, line 165
[user@server gls]$ tail -1 ../gls.log
GLS_PASS
[user@server gls]$

```

Figure 25. Inside one simulation. The testbench loads the program through the serial pins and reads the result on the GPIO pins. At the fast corner every timing message comes from two lines of IHP's memory model.

Those two lines check the memory address against a fixed 1.0 ns setup and hold. IHP's characterized library gives a hold time of 0.32 to 0.34 ns there, and PrimeTime meets it. The model's fixed number is only a placeholder, and every program still passes at the fast corner.

8.2 Scan test

```
bash ~/flash/chip/atpg.sh $RUN
```

TestMAX makes stuck-at patterns for the scan chain. VCS replays every pattern twice, once loaded straight into the flip-flops and once shifted serially through the scan pins as a tester would. Both replays must show 0 mismatches.

```

user@server: ~/flash/runs/test1/signoff/atpg

[user@server atpg]$ grep -E "^(Detected|Possibly|Undetectable|ATPG untestable|Not detected|total faults|test coverage|#internal patterns) " summary.rpt
Detected                DT      112009
Possibly detected       PT        2
Undetectable            UD      639
ATPG untestable         AU      3966
Not detected            ND        22
total faults            116638
test coverage            96.56%
#internal patterns      653
[user@server atpg]$ grep -h "XTB: Simulation" parallel.log fullserial.log | cut -c1-104
XTB: Simulation of 653 patterns completed with 0 mismatches (time: 196300.00 ns, cycles: 1963)
XTB: Simulation of 653 patterns completed with 0 mismatches (time: 121971100.00 ns, cycles: 1219711)
[user@server atpg]$ tail -1 ../../atpg.log
ATPG_DONE
[user@server atpg]$

```

Figure 26. TestMAX's fault summary and both replays. 653 patterns reach 96.56% test coverage, and both replays finish with 0 mismatches.

Test coverage leaves out the 639 faults no test can detect, because no input can make them change an output. The patterns find stuck-at faults, a wire stuck at 0 or 1. They do not reach inside the memories, and they do not test for faults that only make a path slow.

8.3 Clock crossings

```
bash ~/flash/chip/cdc.sh $RUN
```

SpyGlass CDC checks that every signal from the test board enters the clock domain through a synchronizer. It must find 0 unsynchronized crossings.

```

user@server: ~/flash/runs/test1/cdc

[user@server cdc]$ grep -E "Ac_sync0[12]|Ar_sync01|Ar_syncdeassert01" verify.rpt | sed -E 's/^\[[0-9]+\] +([A-Za-z_0-9]+) +Info +[^\s]+ +[0-9]+ +[0-9]+ +/\1: /; s/flash_soc\./g; s/ +/ /g' | cut -c1-104 | head -5
Ac_sync02: Synchronized Crossing: destination flop sclk_s[0], clocked by clk, source primary input sclk,
Ac_sync02: Synchronized Crossing: destination flop sdi_s[0], clocked by clk, source primary input sdi, c
Ac_sync02: Synchronized Crossing: destination flop load_s[0], clocked by clk, source primary input load_
Ar_sync01: Reset signal 'rst_n' for 'clear' pin of flop 'sclk_s[0]', clocked by 'clk', is synchronized b
Ar_sync01: Reset signal 'rst_n' for 'clear' pin of flop 'u_core.core_busy_q[0]', clocked by 'clk', is sy
[user@server cdc]$ grep -c "Ac_unsync" verify.rpt
0
[user@server cdc]$

```

Figure 27. SpyGlass finds a synchronizer on all three loader inputs and on the reset, and 0 unsynchronized crossings.

The test board's loader clock and data pass through separate synchronizers. So whoever builds the board must change the data pin only while the loader clock is low and at least 4 chip clock cycles from either edge. The package's datasheet gives the same rule.

8.4 Voltage drop and electromigration

```

export ANSYSLMD_LICENSE_FILE=<port>@<your Ansys license server>
bash ~/flash/chip/ir/ir.sh $RUN

```

RedHawk-SC checks the power grid in five cases:

- current enters only through the straps next to the supply pads
- current enters through every strap from the pad ring
- the first case again, with the switching activity of the t_work program
- two controls, a weakened grid and limits cut 1000 times, which must look worse

No real case may break an EM limit. The worst drop must stay inside the 10% allowance.

```

user@server: ~/flash/runs/test1

[user@server test1]$ grep worst_cell ir.log | sed -E 's/worst_cell=([0-9.]+) [^ ]+ /worst=\1 V/; s/em_wire_max=/E
M wire %=/; s/em_via_max=/via %=/; s/em_fail=/fails=/'
pads worst=1.173 V EM wire %=62.12 via %=35.64 fails=0
straps worst=1.19 V EM wire %=20.3 via %=12.89 fails=0
saif worst=1.187 V EM wire %=26 via %=17.69 fails=0
emcontrol worst=1.173 V EM wire %=6.212e+04 via %=3.564e+04 fails=5000
defect worst=1.167 V EM wire %=69.98 via %=34.73 fails=0
[user@server test1]$

```

Figure 28. RedHawk-SC's result for each case. It gives the worst cell voltage on the 1.2 V supply and the worst wire and via current as a percent of IHP's limit. The last field counts the failing rows.

Case	Worst cell	Drop	Worst EM
pads only, 10% activity	1.173 V	27 mV, 2.3%	62.1% of the limit
every strap, 10% activity	1.190 V	10 mV	20.3%
pads only, t_work activity	1.187 V	13 mV, 1.1%	26.0%
control, weakened grid	1.167 V	33 mV	must be worse, and is
control, limits cut 1000 times			all 5,000 listed rows fail, as they must

No real case breaks an EM limit, and the worst drop is far inside the allowance. The model leaves out the pad ring's own rails and the package, and it does not check current that changes quickly.

Stage 9: Review records and final signoff

```

cat $RUN/review/SUMMARY.txt $RUN/review/EVIDENCE.txt
python3 ~/flash/chip/review/sign.py $RUN "Your Name"
bash ~/flash/chip/final_signoff.sh $RUN

```

Some things a tool can report but cannot judge. A person reads the evidence and signs a record for each. make_chip.sh ends with unsigned drafts in \$RUN/review and the evidence beside them. Sign only if the evidence agrees with the basis sign.py lists. If SUMMARY.txt names an UNEXPECTED pin, do not sign. Find out why that pin has no clock first.

Record	What the person checks
timing_waivers.json	the two unlocked pins are the memories' unused self-test clocks
timing_coverage.json	every timing check PrimeTime skips has a reason
formality_coverage.json	the one point left out is the scan output, which the scan test covers
atpg/evidence.json	the coverage, the replays and every TestMAX warning

```

user@server: ~/flash/runs/test1

[user@server test1]$ ls review
EVIDENCE.txt
formality_coverage.json
SUMMARY.txt
timing_coverage.json
timing_waivers.json
[user@server test1]$ grep -E "unlocked pins|formality" review/SUMMARY.txt | cut -c1-104
unlocked pins drafted: u_soc/u_dmem/A_BIST_CLK u_soc/u_imem/A_BIST_CLK
formality: passing 2188, excluded 1 ['i:/WORK/flash_chip/scan_out', 'r:/WORK/flash_chip/scan_out'], cloc
formality unmatched: none reported
[user@server test1]$ grep -h "reviewed_by" signoff/timing_waivers.json signoff/timing_coverage.json signoff/formality_coverage.json signoff/atpg/evidence.json
"reviewed_by": "Cesar Valentin",
"reviewed_by": "Cesar Valentin",
"reviewed_by": "Cesar Valentin",
"reviewed_by": "Cesar Valentin",
[user@server test1]$

```

Figure 29. The drafts make_chip.sh leaves for review, the summary that names the two unlocked pins and the one Formality exclusion, and the four records after signing.

The signed records are inputs to the checker, so final_signoff.sh runs every signoff step again. Stage 9 is done when the last line of signoff/CHECK.txt says SIGNOFF CLEAN.

```

user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ for c in slow typ fast slow_shift typ_shift fast_shift; do printf "%-11s" $c; grep -h "^FLASH_PT corner" pt_${c}/pt.log | head -1 | grep -o "setup_wns=[-0-9.]* hold_wns=[-0-9.]*"; done
slow      setup_wns=0.706543 hold_wns=0.349285
typ       setup_wns=7.532767 hold_wns=0.204417
fast      setup_wns=11.438047 hold_wns=0.115814
slow_shift setup_wns=0.706543 hold_wns=0.349287
typ_shift  setup_wns=7.532766 hold_wns=0.204403
fast_shift setup_wns=11.438048 hold_wns=0.115845
[user@server signoff]$ for c in slow typ fast; do grep -h "^FLASH_PT_NOISE" pt_${c}/pt.log; done
FLASH_PT_NOISE corner=slow worst_slack=0.03 violators=0
FLASH_PT_NOISE corner=typ worst_slack=0.05 violators=0
FLASH_PT_NOISE corner=fast worst_slack=0.03 violators=0
[user@server signoff]$ grep -c VIOLATED pt_slow/violators.rpt pt_typ/violators.rpt pt_fast/violators.rpt
pt_slow/violators.rpt:0
pt_typ/violators.rpt:0
pt_fast/violators.rpt:0
[user@server signoff]$

```

Figure 30. PrimeTime after the final signoff. Every run passes with 0 noise violators and 0 electrical violations.

```

user@server: ~/flash/runs/test1/signoff

[user@server signoff]$ grep -E "(fc_flow|chip_drc|chip_precheck|chip_lvs_completion|formality|atpg_coverage|formality_coverage|electrical_slow|pt_slow|noise_slow|timing_waivers|pt_report_analysis_coverage_slow) " CHECK.txt
| cut -c1-104
fc_flow                PASS      done=True errors=0
pt_slow                PASS      period=20.0ns setup_wns=0.707 hold_wns=0.349 spef_nets=
noise_slow            PASS      worst_slack=0.03 violators=0
formality              PASS      SUCCEEDED Passing=2188
chip_drc               PASS      klayout violations=0 recommended_only=0
chip_precheck          PASS      klayout violations=0 recommended_only=0
electrical_slow        PASS      unwaived=0 waived=0
pt_report_analysis_coverage_slow PASS      reviewed reasons=conditional_arc:304,constant_at_pin:6,
formality_coverage     PASS      passing=2188 excluded=1 clock_gate_latches=0; requires
atpg_coverage          PASS      patterns=653 test_coverage=96.56 total_faults=116638 St
chip_lvs_completion    PASS      latest match verdict and successful exit required
[user@server signoff]$ grep -c " PASS " CHECK.txt
139
[user@server signoff]$ tail -1 CHECK.txt
SIGNOFF CLEAN: 139 of 139 checks pass
[user@server signoff]$

```

Figure 31. A sample of the checker's rows after the final signoff, the count of passing checks, and its last line: SIGNOFF CLEAN, 139 of 139.

Stage 10: The release package

```
bash ~/flash/chip/package/package.sh $RUN
```

This fills IHP's release folder in \$RUN/package/IHP__RVSoC8787. It saves the GDS with the settings IHP asks for and renames the top cell to the IP name. It compares that GDS with the signed-off chip, shape for shape and label for label, and runs IHP's pre-check on it. It passed if it ends with PACKAGE_OK. The package's doc/Datasheet.md gives the bring-up steps for the built chip.

```

user@server: ~/flash/runs/test1/package

[user@server package]$ ls IHP__RVSoC8787 IHP__RVSoC8787/release/v.1.0.0/gds IHP__RVSoC8787/release/v.1.0.0/netlist
IHP__RVSoC8787:
doc
README.md
release
RVSoC8787-main

IHP__RVSoC8787/release/v.1.0.0/gds:
RVSoC8787.gds

IHP__RVSoC8787/release/v.1.0.0/netlist:
flash_chip.v
RVSoC8787.cdl
[user@server package]$ grep -h "^LAYERS\|^TEXTS" xor.txt texts.txt
LAYERS 59 DIFFERING 8
TEXTS 1676812 1676812 ONLY_A 0 ONLY_B 0
[user@server package]$ tail -3 ../package.log | sed "s#/#ecel/UFAD/c.valentin/tt#~#"
PRECHECK_EXIT=0
PRECHECK_MARKERS=0
PACKAGE_OK ~/flash/runs/test1/package/IHP__RVSoC8787
[user@server package]$

```

Figure 32. The release folder, the shape and label comparison with the signed-off chip, and the end of the package log. All 1,676,812 labels match, IHP's pre-check finds no markers, and the script ends with PACKAGE_OK.

The shape log lists eight layers as differing. Each holds only labels, with 0 area and 0 polygons of difference, so the geometry is the same.

Results of the fresh build

These are the results of test1, the fresh build of the repository that every screenshot comes from. A new build on the same tools should land within a few hundredths of a nanosecond of these.

Check	Result
final signoff	SIGNOFF CLEAN, 139 of 139 checks
slow setup slack	+0.707 ns at the 20 ns period
hold slack, slow, typ and fast	+0.349, +0.204 and +0.116 ns, in both modes
noise and electrical limits	0 violations at every corner
Formality	2,188 compare points pass
chip DRC and IHP pre-check	0 and 0
block and whole-chip LVS	match
metal density	44.6% to 55.0% on every metal layer
power	3.45 mW at the typical corner
programs on the gates	all pass at slow, typ and fast, and the control fails
scan test	653 patterns, 96.56% test coverage, 0 mismatches
clock crossings	0 unsynchronized
worst static supply drop	27 mV, 2.3%
worst EM	62.1% of IHP's limit
die size	1708 by 1753 μm

This signoff leaves three things unproven. IHP publishes no factory-qualified worst and best wire models. LVS does not check the insides of IHP's memory and pad cells. The voltage-drop check is static only. The tutorial explains each limit.

Command summary

```
git clone https://github.com/XxMasterepicxX/ibex-sg13g2-chip.git ~/flash
bash ~/flash/setup/setup.sh                               once, about two hours
bash ~/flash/chip/make_chip.sh my_chip                     stages 1 to 8
R=~/.flash/runs/my_chip
python3 ~/flash/chip/review/sign.py $R "Your Name"        after reading the drafts
bash ~/flash/chip/final_signoff.sh $R                     ends with SIGNOFF CLEAN
bash ~/flash/chip/package/package.sh $R                   ends with PACKAGE_OK
```